Support Ticket Triage Agent — System Design

Sep 28, 2026 · @hardik chhatwal

Overview

The system turns one incoming support ticket into one validated routing decision: a category, a priority, a destination team, and whether to auto-resolve from the knowledge base or escalate to a human.

It is a ReAct agent over a retrieval-augmented (RAG) knowledge base. The output is a strongly typed `TicketDecision` object that a downstream ticketing system can consume directly:

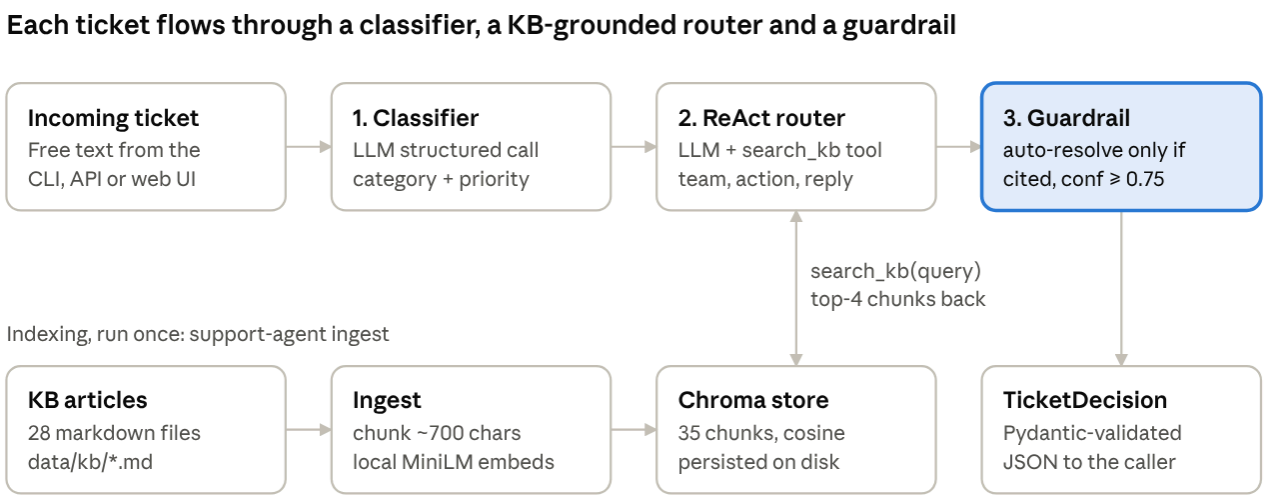
```
{
  "category": "account",
  "priority": "high",
  "team": "account_management",
  "action": "auto_resolve",
  "kb_citation": "reset_password",
  "confidence": 0.92,
  "draft_resolution": "On the sign-in page click 'Forgot password'...",
  "reasoning": "The reset_password article answers this ticket directly."
}
```

In scope: one ticket at a time; three bounded decisions (classify, route, resolve-or-escalate); a drafted reply only when grounded in a cited KB article.

Out of scope: multi-turn conversations with customers, open-ended replies not grounded in the KB, ticket lifecycle or state management, and sending anything to customers. The system recommends; people and the ticketing system act.

Architecture

The pipeline is two LLM calls and one deterministic check, in a fixed order. Retrieval is the only loop: the router may search the KB several times before it decides.



triage pipeline · 3 per-ticket stages, 1 offline indexing path

What happens to one ticket:

1. **Classify.** The classifier agent reads the ticket and returns a `TicketClassification` (category, priority, one-sentence reasoning).
2. **Retrieve and reason.** The router agent receives the ticket plus its category and priority. It must call `search_kb` at least once; each call embeds the query and returns the top 4 KB chunks with similarity scores.
3. **Decide.** The router returns a `RoutingDecision`: team, action, cited article, confidence, draft reply and reasoning.
4. **Guard.** Plain Python code downgrades any auto-resolve that lacks a citation or has confidence below 0.75 to an escalation.
5. **Merge and return.** The classification and routing outputs are merged into one validated `TicketDecision` and returned as JSON.

Components

Five components, each in its own module under `src/support_agent/`.

Component	Module	Responsibility	Key details
Decision schema	`schema.py`	The contract every other part produces or consumes	Enums for 5 categories, 4 priorities, 5 teams, 2 actions; confidence bounded 0–1; invalid output raises a validation error
Knowledge base	`knowledge_base.py`	Index and search KB articles (the RAG layer)	Chroma collection, cosine distance; paragraph-aware chunks of ⩽700 characters; local all-MiniLM-L6-v2 embeddings, so no API key is needed

Component	Module	Responsibility	Key details
Classifier	`agent.py`	Assign category and priority	Pydantic AI agent with `output_type=TicketClassification`, no tools
ReAct router	`agent.py`	Choose team, decide auto-resolve vs escalate, draft a grounded reply	Pydantic AI agent with the `search_kb` tool and `output_type=RoutingDecision`; prompt says "when in doubt, escalate"
Safety guardrail	`agent.py`	Enforce the auto-resolve policy deterministically	Downgrades auto-resolve to escalate if no citation or confidence < 0.75; appends the reason to the reasoning field

Schema values

- Categories: billing, technical, account, feature_request, general.
- Priorities: low, medium, high, urgent.
- Teams: billing, technical_support, account_management, product, general_support.
- Actions: auto_resolve, escalate.

`TriageAgent` wires the parts together. `triage()` returns the decision; `triage_explained()` also returns the classifier's reasoning, every KB search with its hits, and whether the guardrail fired.

Technology choices

The stack is deliberately small: one agent framework, one LLM provider, one local vector store.

Concern	Choice	Why	Rejected alternative
Agent loop + output schema	Pydantic AI (0.8)	Its native tool-calling loop is the ReAct loop, and `output_type` enforces the schema, so malformed output fails loudly instead of mis-routing	smolagents (a second framework for the same loop); LangGraph (no branching state to manage)
LLM	OpenAI `gpt-5.4-mini` (configurable)	Strong tool calling and structured output at low latency (~4 s per ticket for both calls)	Local models (weaker structured output; not required by the brief)

Concern	Choice	Why	Rejected alternative
Vector store	Chroma, persisted locally	Near-zero setup, persistent on disk, enough for a KB of this size	Hosted vector DBs (unneeded operational overhead)
Embeddings	all-MiniLM-L6-v2 via ONNX (Chroma default)	Runs locally, reproducible, no API key or cost	`text-embedding-3-small` (a possible upgrade; see Future work)
HTTP layer	FastAPI + Uvicorn	Typed request/response models reuse the Pydantic schema; OpenAPI docs for free	Flask (no built-in validation)
Web UI	One static HTML page, vanilla JS	No build step; served by the same API	React/Streamlit (extra dependencies for a test console)

Interfaces

The same `TriageAgent` is reachable three ways: a CLI, an HTTP API, and a web UI served by the API.

CLI (`support-agent`)

Command	What it does
`support-agent ingest`	Parses, chunks, embeds and indexes `data/kb/*.md` into Chroma
`support-agent triage "<text>"`	Triages one ticket (also `--file` or stdin) and prints the decision JSON
`support-agent serve`	Starts the API and web UI on http://127.0.0.1:8000

HTTP API (FastAPI; interactive docs at `/docs`)

Method	Path	Returns
POST	`/triage`	`TicketDecision`, the contract for downstream systems

Method	Path	Returns
POST	`/triage/explain`	`TriageTrace`: the decision plus classifier reasoning, KB searches with scores, and a guardrail flag
GET	`/kb`, `/kb/{id}`	KB articles
GET	`/tickets`	The labeled evaluation set
GET	`/health`	Model name, whether the API key is set, KB article and chunk counts

Requests take `{"text": "..."}` (1–10,000 characters). Empty text returns 422, a missing key or empty KB returns 503, and a model or provider failure returns 502 with the error.

Web UI (three tabs)

- **Triage:** the decision, confidence against the threshold, the draft reply with its source article, and a step-by-step trace of how the agent decided.
- **Test set:** runs labeled tickets and shows predicted vs expected labels with live metric tiles.
- **Knowledge base:** search and read the KB articles.

Safety and reliability

A wrong auto-resolve (telling a customer their problem is solved when it isn't) costs far more than an unnecessary escalation, so the design biases every layer toward escalating.

1. **Prompt policy.** The router may auto-resolve only when a retrieved article "directly and completely" answers the ticket. It is told to escalate on partial matches, ambiguous or multi-issue tickets, billing disputes, account recovery without backup codes, persistent bugs and urgent incidents.
2. **Grounding requirement.** An auto-resolve must cite a KB `article_id`, and the draft reply must come from that article.
3. **Deterministic guardrail.** Code, not the model, enforces the policy. Any auto-resolve without a citation or with confidence below 0.75 (configurable) becomes an escalation, and the draft reply is removed.
4. **Schema validation.** Pydantic rejects out-of-range values (for example an unknown team or confidence above 1), so a malformed model response fails instead of silently mis-routing.

5. **Transparent reasoning.** Every decision carries the model's reasoning, and `/triage/explain` exposes the retrieval evidence, so a reviewer can audit why a ticket was resolved.

Failure handling. API errors surface as HTTP 502 with the cause rather than a partial decision. The server starts without a key, so `/health` can report a missing key before any ticket is triaged.

Configuration, deployment and secrets

All settings come from environment variables, loaded from a git-ignored `.env` file.

Variable	Default	Purpose
`OPENAI_API_KEY`	none (required)	Authenticates the classifier and router calls
`SUPPORT_AGENT_MODEL`	`openai:gpt-4o-mini` (set to `openai:gpt-5.4-mini` in this deployment)	Model for both LLM calls
`SUPPORT_AGENT_AUTORESOLVE_THRESHOLD`	0.75	Minimum confidence for an auto-resolve
`SUPPORT_AGENT_TOP_K`	4	KB chunks returned per search
`SUPPORT_AGENT_CHROMA_DIR`	`data/chroma`	Where the index is persisted

Deployment. Python 3.9+. Run `pip install -e .`, then `support-agent ingest` once, then `support-agent serve`. The server binds to 127.0.0.1 by default; use `--host 0.0.0.0` behind a reverse proxy to expose it.

Secrets. The API key lives only in `.env`, which `.gitignore` excludes; the repository ships `.env.example` with a placeholder. The public repository was scanned: no keys, tokens or personal data are committed. The local Chroma index is also excluded and is rebuilt with `ingest`.

Not yet built for production: authentication on the API, rate limiting, and request logging.

Testing

Two layers: 22 offline unit and integration tests that need no API key, and a live evaluation against the labeled ticket set.

Suite	Covers	LLM used
`test_schema.py`	Enum values, confidence bounds, merging classification and routing	None
`test_knowledge_base.py`	Article parsing, chunking, ingest, retrieval ranking against the real KB	None (real local embeddings)
`test_agent.py`	Guardrail rules; full pipeline wiring; a scripted ReAct run that searches, cites and auto-resolves; the explain trace	Pydantic AI `TestModel` / `FunctionModel`
`test_api.py`	Every endpoint, validation errors, the served web page	`TestModel` injected into the app

Chroma retrieval runs for real in every test, so retrieval regressions are caught offline.

The live evaluation (`python eval/evaluate.py`) runs the real model on all 76 labeled tickets and reports the spec metrics. Its method and targets are in the companion *Problem Definition, Data Processing & Evaluation Criteria* document.

Limitations and future work

Limitation	Impact	Next step
Small, synthetic KB (28 articles) and ticket set (76)	Metrics are indicative, not statistically strong	Evaluate on a larger public support-routing dataset; the evaluator already accepts any JSONL with the same fields
Model-reported confidence is not calibrated	The 0.75 threshold may be too loose or too strict	Calibrate the threshold on held-out data, or add retrieval score as a second signal

Limitation	Impact	Next step
One category and team per ticket	Multi-issue tickets are forced into one route, then escalated	Allow multiple labels, or split tickets upstream
Local MiniLM embeddings	Weaker recall on paraphrased or jargon-heavy tickets	Try `text-embedding-3-small`, or hybrid keyword + vector search
Two sequential LLM calls (~4 s per ticket)	Latency and cost scale with volume	Merge classification into the router call, or cache repeated questions
No API authentication or rate limiting	Safe only on localhost or behind a gateway	Add API keys and rate limits before exposing the service